CIS 3370 · INTRUSION DETECTION SYSTEMS · LAB B OF 3

Zeek, from a capture to Elasticsearch

Lab A shipped what a host writes about itself. This lab ships what the network says. Zeek reads packets and writes one record per connection, DNS query, web request and TLS handshake. You will run it on the Lecture 6 capture twice, once in its default text format and once in JSON, and measure the difference. Then you will ship the records to Elasticsearch as raw lines and through Filebeat's Zeek module, find the port scan hidden in the capture, and finally point Zeek at `zeek-lab`'s own network card while you make traffic and `client-01` scans it. Every number and screenshot here comes from running these steps on the class stack.

1. Install 2. A capture, two formats 3. Ship it two ways 4. Find the scan 5. Zeek, live

0 Before you start

Finish Lab A first. This lab reuses its pieces:

- **The header file and two short names** in your host terminal, `ES=http://localhost:9200` and `KEYFILE=@$HOME/.elastic/auth.hdr`.
- **The `beats` user** and its `filebeat_publisher` role.
- **The `filebeat` data view** in Kibana.

Copy the generate bundle into the `zeek-lab` data folder, the same way you copied it for `client-01`.

`HOST` On a Mac or Linux:

```
cp ~/Downloads/cis3370-generate-bundle.zip ~/docker-elk/zeek-lab/data/
```

`HOST` On Windows with WSL:

```
cp /mnt/c/Users/YOU/Downloads/cis3370-generate-bundle.zip \
  ~/docker-elk/zeek-lab/data/
```

1 Install Zeek and Filebeat on zeek-lab

Ubuntu has no Zeek package of its own. The Zeek project publishes one for Ubuntu 26.04 on both processor types. `zeek-core` is the analysis engine alone, which is all this lab needs.

HOST `docker compose exec zeek-lab bash`, then in ZEEK-LAB :

```
apt-get update
apt-get install -y curl gnupg ca-certificates unzip nano
REPO=download.opensuse.org/repositories/security:/zeek/xUbuntu_26.04
echo "deb http://$REPO/ /" > /etc/apt/sources.list.d/zeek.list
curl -fsSL https://$REPO/Release.key \
  | gpg --dearmor > /etc/apt/trusted.gpg.d/zeek.gpg
apt-get update
apt-get install -y zeek-core
/opt/zeek/bin/zeek --version
```

```
/opt/zeek/bin/zeek version 9.0.1
```

Then Filebeat, exactly as in Lab A:

```
cd /tmp
ARCH=$(dpkg --print-architecture)
URL=https://artifacts.elastic.co/downloads/beats/filebeat
curl -L -O $URL/filebeat-9.5.3-${ARCH}.deb
dpkg -i filebeat-9.5.3-${ARCH}.deb
filebeat version
```

2 One capture, two formats

`capture.pcap` is the minute from Lecture 6 in which `nmap -sT` scanned 1,000 ports on each of the machine's two addresses while eight web pages loaded. Run Zeek on it in its default format first.

ZEEK-LAB

```
cd /home/ubuntu/data
unzip -o cis3370-generate-bundle.zip
mkdir -p zeek-tsv && cd zeek-tsv
/opt/zeek/bin/zeek -C -r ../cis3370-generate-bundle/capture.pcap
ls
head -8 conn.log
```

```
conn.log  dns.log  packet_filter.log  ssl.log

#separator \x09
#set_separator  ,
#empty_field    (empty)
#unset_field    -
#path   conn
#open   ...
#fields ts      uid     id.orig_h       id.orig_p       id.resp_h
id.resp_p       proto   service duration        ...
#types  time    string  addr    port    addr    port    enum    string
interval        ...
```

PART	MEANING
`-r capture.pcap`	Read packets from a file instead of a network card
`-C`	Ignore checksums. A container's own outgoing packets are captured before the network card fills in the checksum; without this flag Zeek throws them away as damaged.

This is TSV, tab-separated values. Every record is one line of values, and the eight lines starting with `#` say what the columns are, what type each is, and how a missing value is written (`-`). A program reading this file must read those header lines first, or the values mean nothing. Now the same capture in JSON:

```
cd /home/ubuntu/data
mkdir -p zeek-logs && cd zeek-logs
/opt/zeek/bin/zeek -C -r ../cis3370-generate-bundle/capture.pcap \
  LogAscii::use_json=T
wc -l *.log
head -1 conn.log
ls -l ../zeek-tsv/conn.log conn.log
```

```
  2024 conn.log
    16 dns.log
     1 packet_filter.log
     8 ssl.log
{"ts":1793423385.033747,"uid":"Cwwanq4ujgbm2X05Ag","id.orig_h":"172.21.0.5","id.
"id.resp_h":"216.59.63.135","id.resp_p":443,"proto":"tcp","service":"ssl",
"duration":0.30289793014526367,"orig_bytes":2454,"resp_bytes":207202,"conn_state
...}
-rw-r--r-- 1 root root 227946 ... ../zeek-tsv/conn.log
-rw-r--r-- 1 root root 719350 ... conn.log
```

The same 2,024 connections, measured side by side:

QUESTION	TSV (DEFAULT)	JSON (`USE_JSON=T`)
How do you know what a value is?	Count columns against the `#fields` line	Every value carries its name
Lines in `conn.log`	2,033 (8 header, 2,024 records, 1 footer)	2,024, one per record
A missing value	`-`, which a reader must know to skip	The field is simply absent
True and false	`T` and `F`	`true` and `false`
The first connection's duration	`0.302898`	`0.30289793014526367`
Size of `conn.log`	227,946 bytes	719,350 bytes, 3.2 times bigger
Read by Filebeat's `zeek` module?	No	Yes, it requires JSON

JSON costs disk, and buys records that every tool can read without knowing Zeek's header conventions, one line at a time, with the full precision Zeek measured. For a log you ship somewhere else, that trade is almost always worth it.

Zeek invents a new `uid` each run, so yours differ. Every other value comes from the packets and matches.

3 Ship the records two ways

Secrets first

`ZEEK-LAB` The beats password, typed once and never shown:

```
filebeat keystore create
read -rsp "Password for beats: " PW; echo
printf '%s' "$PW" | filebeat keystore add ES_PWD --stdin
unset PW
```

`HOST` The admin key, for setup only:

```
sed 's/^Authorization: ApiKey //' ~/.elastic/auth.hdr \
  | tr -d '\n' | base64 -d \
  | docker compose exec -T zeek-lab filebeat keystore add ADMIN_KEY --stdin
```

Way one: conn.log as a raw stream

`ZEEK-LAB` The same `filestream` input as Lab A, plus the `ndjson` parser, which turns each JSON line into fields. `target` keeps them under one name so they cannot collide with anything else.

```
cp /etc/filebeat/filebeat.yml /etc/filebeat/filebeat.yml.original
cat > /etc/filebeat/filebeat.yml <<'EOF'
filebeat.inputs:
  - type: filestream
    id: zeek-conn-raw
    paths:
      - /home/ubuntu/data/zeek-logs/conn.log
    parsers:
      - ndjson:
          target: "zeek_raw"
    fields:
      lab: zeek-stream

filebeat.config.modules:
  path: ${path.config}/modules.d/*.yml
  reload.enabled: false

output.elasticsearch:
  hosts: ["http://elasticsearch:9200"]
  username: "beats"
  password: "${ES_PWD}"
EOF
```

Way two: the zeek module

```
filebeat modules enable zeek
nano /etc/filebeat/modules.d/zeek.yml
```

The file lists 39 filesets, one per kind of Zeek log, all switched off. Switch on four and give each two paths: the capture's logs now, and the live logs of section 5 later. The Zeek file is `conn.log`, but its fileset is called `connection`. Make these four blocks look exactly like this, then save with Ctrl+O, Enter, and leave with Ctrl+X:

```
  connection:
    enabled: true
    var.paths:
      - /home/ubuntu/data/zeek-logs/conn.log
      - /home/ubuntu/data/zeek-live/conn.log
  dns:
    enabled: true
    var.paths:
      - /home/ubuntu/data/zeek-logs/dns.log
      - /home/ubuntu/data/zeek-live/dns.log
  http:
    enabled: true
    var.paths:
      - /home/ubuntu/data/zeek-logs/http.log
      - /home/ubuntu/data/zeek-live/http.log
  ssl:
    enabled: true
    var.paths:
      - /home/ubuntu/data/zeek-logs/ssl.log
      - /home/ubuntu/data/zeek-live/ssl.log
```

Check the edit, test, load the template and every Zeek pipeline with the admin key, then take the key back off the machine and start Filebeat:

```
grep -A4 -E "^  (connection|dns|http|ssl):" /etc/filebeat/modules.d/zeek.yml
filebeat test config
filebeat setup --index-management --pipelines \
  --modules zeek --force-enable-module-filesets \
  -E output.elasticsearch.username=null \
  -E output.elasticsearch.password=null \
  -E 'output.elasticsearch.api_key=${ADMIN_KEY}'
filebeat keystore remove ADMIN_KEY
nohup filebeat -e > /var/log/filebeat-run.log 2>&1 &
```

```
Config OK
Index setup finished.
Loaded Ingest pipelines
successfully removed key: ADMIN_KEY
```

`--force-enable-module-filesets` loads the pipelines for all 39 filesets, so switching another one on later needs no new setup.

Validate: every line arrived, and arrived once per way

CONSOLE

```
GET filebeat-*/_search?filter_path=aggregations
{
  "size": 0,
  "aggs": {
    "datasets": {
      "terms": { "field": "event.dataset" }
    }
  }
}

GET filebeat-*/_count?q=fields.lab:zeek-stream
```

CURL

HOST

```
curl -s -H "$KEYFILE" "$ES/filebeat-*/_count?
q=event.dataset:zeek.connection"
curl -s -H "$KEYFILE" "$ES/filebeat-*/_count?q=fields.lab:zeek-stream"
```

WHAT	LINES ZEEK WROTE	DOCUMENTS
Raw stream of `conn.log`	2,024	2,024
`zeek.connection`	2,024	2,024
`zeek.dns`	16	16
`zeek.ssl`	8	8

A missing line is a lost connection, and an IDS that silently loses records is worse than one that is down, because nobody knows to look. Check every count.

The same connection, both ways

Take the `uid` from `head -1 conn.log` and put it on the first line.

CONSOLE

```
GET filebeat-*/_search?q=zeek_raw.uid:Cwwanq4ujgbm2X05Ag

GET filebeat-*/_search?q=zeek.session_id:Cwwanq4ujgbm2X05Ag

GET filebeat-*/_mapping/field/zeek_raw.ts,zeek_raw.id.orig_h,source.ip
```

CURL

```
ZUID=Cwwanq4ujgbm2X05Ag
curl -s -H "$KEYFILE" -G "$ES/filebeat-*/_search" -d pretty \
  -d q=zeek_raw.uid:$ZUID -d _source=@timestamp,zeek_raw
curl -s -H "$KEYFILE" -G "$ES/filebeat-*/_search" -d pretty \
  -d q=zeek.session_id:$ZUID \
  -d _source=@timestamp,source,destination,network,zeek.connection
```

QUESTION	RAW STREAM	ZEEK MODULE
When did it happen?	`@timestamp` is when Filebeat read the line. The real time hides in `zeek_raw.ts`, a plain number.	`@timestamp` is the connection's start
Who connected?	`zeek_raw.id.orig_h`, stored as `keyword`, text	`source.ip`, stored as `ip`, so `172.21.0.0/16` works as a search
Field names	Zeek's: `id.resp_p`, `conn_state`	ECS, the names every Elastic tool expects: `destination.port`, `zeek.connection.state`
Anything added?	No	Country and network owner of the outside address, a `network.community_id`, and `state_message: "Normal establishment and termination."`

4 Find the scan

CONSOLE

```
GET filebeat-*/_search?filter_path=aggregations
{
  "size": 0,
  "query": { "term": { "event.dataset": "zeek.connection" } },
  "aggs": {
    "state": { "terms": { "field": "zeek.connection.state" } },
    "span":  { "stats": { "field": "@timestamp" } },
    "rejected": {
      "filter": { "term": { "zeek.connection.state": "REJ" } },
      "aggs": {
        "ports":   { "cardinality": { "field": "destination.port" } },
        "targets": { "terms": { "field": "destination.ip" } }
      }
    }
  }
}
```

CURL

Paste the same body into a file, say `/tmp/scan.json`, then:

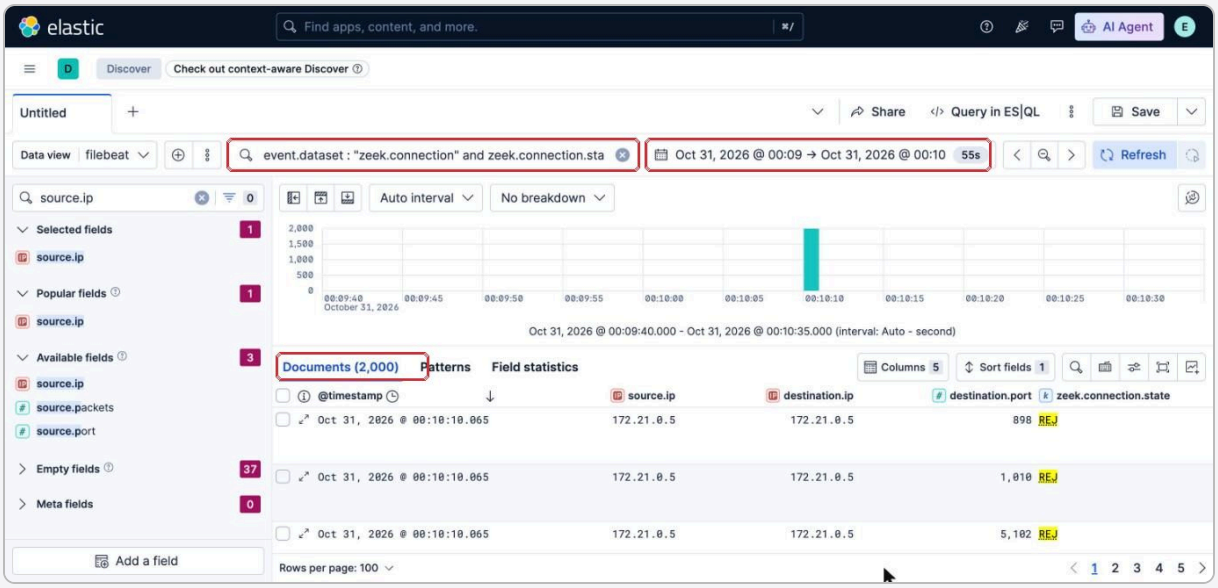
```
curl -s -H "$KEYFILE" -H 'Content-Type: application/json' \
  "$ES/filebeat-*/_search?filter_path=aggregations&pretty" -d
@/tmp/scan.json
```

STATE	MEANING	CONNECTIONS
`REJ`	Asked to connect and was refused at once	2,000
`SF`	Opened and closed normally	8
`S0`	Asked, and never got an answer	8
`SHR`	Zeek saw only the reply side	8

The 2,000 refusals hit **1,000 distinct ports**: 1,000 on `127.0.0.1` and 1,000 on `172.21.0.5`, the two addresses the scan named. The rest of the capture is 24 connections, the eight page loads and the lookups behind them. The capture spans 43.7 seconds; the `span` answer gives its first and last moment.

DISCOVER

Set the time picker to an absolute range from the `span` minimum to its maximum, search `event.dataset : "zeek.connection" and zeek.connection.state : "REJ"`, and add `source.ip`, `destination.ip`, `destination.port` and `zeek.connection.state` as columns. All 2,000 fall in the same second.



2,000 refused connections inside one second. Kibana shows times in your browser's time zone.

5 Zeek, live, on its own network card

So far Zeek read a recording. Now it watches `zeek-lab`'s network card while you make traffic, and Filebeat ships each record as Zeek writes it. The module already lists the `zeek-live` folder, so Filebeat needs no change: when the files appear, it finds them within about 10 seconds.

ZEEK-LAB

```
mkdir -p /home/ubuntu/data/zeek-live && cd /home/ubuntu/data/zeek-live
nohup /opt/zeek/bin/zeek -C -i eth0 LogAscii::use_json=T > zeek.out 2>&1 &
hostname -i
```

`-i eth0` means listen on the network card. Note the address. Then restart Filebeat, so that its own connection to Elasticsearch begins while Zeek is watching. Zeek can only decode a conversation it saw start; a connection opened before Zeek started is just bytes to it.

```
pkill filebeat
nohup filebeat -e > /var/log/filebeat-run.log 2>&1 &
```

Now make traffic from `zeek-lab` itself:

```
curl -s -o /dev/null -w "%{http_code}\n" http://example.com/
curl -s -o /dev/null -w "%{http_code}\n" https://www.elastic.co/
apt-get update
```

`HOST` `docker compose exec client-01 bash`, then in `CLIENT-01` :

```
apt-get install -y nmap
nmap -sT -p 1-1000 zeek-lab
hostname -i
```

```
Not shown: 1000 closed tcp ports (conn-refused)
...
Nmap done: 1 IP address (1 host up) scanned in 0.57 seconds
```

`ZEEK-LAB` Stop Zeek politely, so it writes out everything it holds, then count:

```
pkill -INT zeek
wc -l *.log
grep -o '"message":"[0-9]* packets received[^"]*' reporter.log
```

```
  1018 conn.log
    25 files.log
    25 http.log
     1 packet_filter.log
     2 reporter.log
     1 ssl.log
"message":"3140 packets received on interface eth0, 0 (0.00%) dropped, 0
(0.00%) not processed
```

Check the dropped count every time you capture: a capture that drops packets misses connections.

What Elasticsearch got

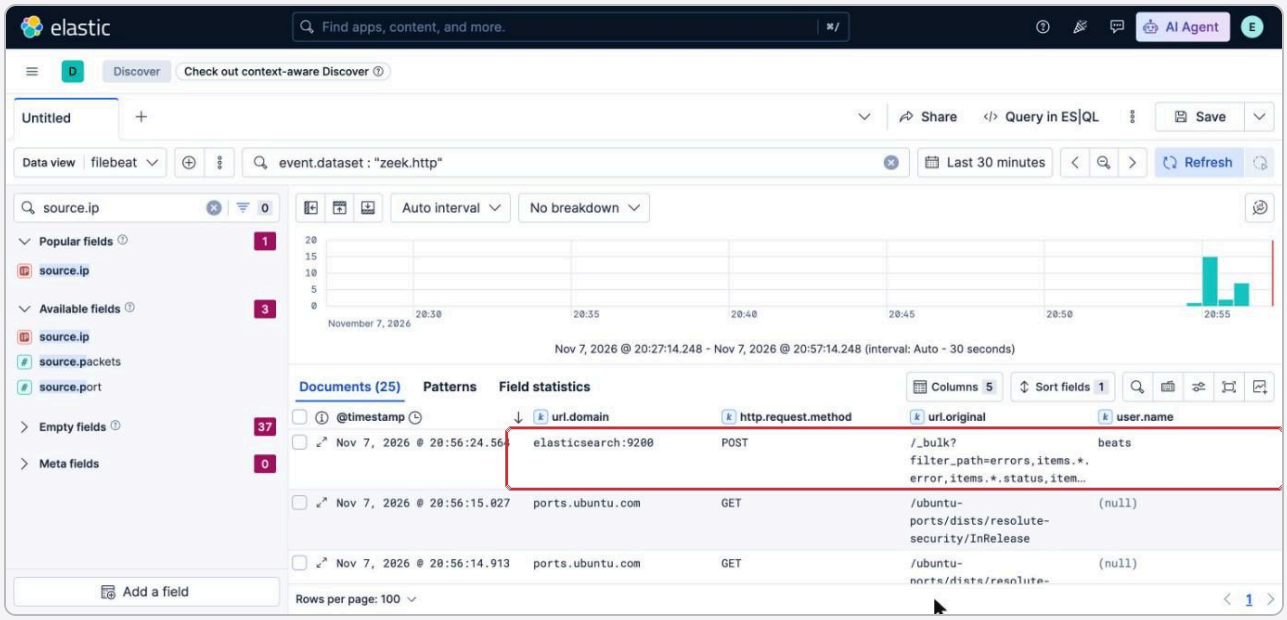
CONSOLE

```
GET filebeat-*/_search?filter_path=hits.total,aggregations
{
  "size": 0,
  "query": {
    "prefix": { "log.file.path": "/home/ubuntu/data/zeek-live/" }
  },
  "aggs": {
    "datasets": { "terms": { "field": "event.dataset" } },
    "from":     { "terms": { "field": "source.ip" } },
    "sites":    { "terms": { "field": "url.domain" } }
  }
}
```

On the class stack, with `zeek-lab` at 172.21.0.5 and `client-01` at 172.21.0.7: 1,044 documents, made of 1,018 `zeek.connection`, 25 `zeek.http` and 1 `zeek.ssl`. 1,000 connections came from `client-01`, all `REJ`: the scan, seen by its target a few seconds after it happened. The capture's bundle counts did not change; Filebeat's registry knows it already sent those files.

Three things the live capture teaches

- **No `dns.log` at all.** Docker answers name lookups inside the container, at `127.0.0.11`, over the loopback interface. The queries never cross `eth0`, so a sensor on `eth0` cannot see them. Where you put a sensor decides what it can see.
- **`https://` is one `ssl` record; `http://` is a full `http` record.** For the encrypted site Zeek knows the server name and certificate. For `example.com` it knows the method, the path, the status code and the user agent.
- **Zeek watched Filebeat ship the logs.** On the class stack 19 of the 25 `http` records were Filebeat talking to `elasticsearch:9200`, and each one carried `user.name: beats`. Skip the Filebeat restart and they are missing, because Zeek never saw that connection begin. This stack speaks plain HTTP, so the account name, and the password with it, cross the network in the clear.



Live http records. The top row is Filebeat itself, POSTing to _bulk as beats.

6 Questions

QUESTION 1

Give two reasons to ship Zeek in JSON and one reason not to, each with a number you measured.

QUESTION 2

A detection rule counts refused connections per minute. What would it report on the raw stream, given where its `@timestamp` comes from? Why is that dangerous?

QUESTION 3

The capture shows 2,000 refusals and the live scan 1,000, from the same kind of nmap command. Explain the difference.

QUESTION 4

Choose a threshold: how many `REJ` from one source in 60 seconds should raise an alert? Justify it with the 24 ordinary connections in the capture, and name a scan your threshold would miss.

QUESTION 5

Your sensor saw no DNS. Where would you put it to see every lookup the lab machines make, and what would it still miss?

QUESTION 6

Zeek recorded `user.name: beats` in Filebeat's own traffic. What exactly would an attacker on this network learn, and what single change to the stack stops it?

7 What to hand in

1. The TSV and JSON `conn.log` sizes, and the first four lines of the TSV header.
2. The count table from section 3, with your own numbers.
3. Your Discover screenshot of the scan.
4. Your live counts: the `wc -l` output, the dropped-packets line, and the datasets from Elasticsearch.
5. Your answers to the six questions.

8 Start over

`ZEEK-LAB`

```
pkill filebeat
pkill zeek
rm -rf /var/lib/filebeat/registry /home/ubuntu/data/zeek-live
```

The Elasticsearch side is shared with Lab A; its start-over section removes it.

Every number and screenshot on this page comes from running these steps on the class stack: Zeek 9.0.1, Filebeat and Elasticsearch 9.5.3, Ubuntu 26.04 containers. Addresses, uids and the packages `apt-get update` fetches differ by machine. The capture's counts should not.